

Module 7

Python – Collections, Functions and Modules

1. Accessing List

Definition: A list is an ordered collection in Python that can store multiple values of different data types such as integers, strings, floats, and booleans. Lists are created using square brackets `[]`.

Theory: Lists allow storing multiple values in a single variable. Elements can be accessed using Slicing is used to access a range of elements.

Lab Program 1: Create a list with elements of multiple data types.

```
my_list = [10, "Python", 3.14, True]

print(my_list)
```

Lab Program 2: Access elements at different index positions.

```
my_list = [10, "Python", 3.14, True]
print(my_list[0])
print(my_list[1])
print(my_list[-1])
```

Practical Example 1: Create a list of multiple data types.

```
data = [5, "Hello", 7.5, False]
print(data)
```

Practical Example 2: Find the length of a list.

```
numbers = [1,2,3,4,5]
print(len(numbers))
```

2. List Operations

Definition: List operations allow users to modify list elements using operators and built-in methods such as `append()`, `insert()`, `remove()`, and `pop()`.

Theory: Common list operations include concatenation (+), repetition (*), and membership (in). Elements can be added using `append()` or `insert()` and removed using `remove()` or `pop()`.

Lab Program 3: Add elements to a list using `insert()` and `append()`.

```
numbers = [1,2,3]
numbers.append(4)
```

```
numbers.insert(1,10)
print(numbers)
```

Lab Program 4: Remove elements using pop() and remove().

```
numbers = [10,20,30,40]
numbers.pop()
numbers.remove(20)
print(numbers)
```

Practical Example 3: Update a list using insert() and append().

```
my_list = [5,6,7]
my_list.append(8)
my_list.insert(0,4)
print(my_list)
```

Practical Example 4: Remove elements using pop() and remove().

```
my_list = [10,20,30,40]
my_list.pop(2)
my_list.remove(10)
print(my_list)
```

3. Working with Lists

Definition: Working with lists means performing operations like iterating, sorting, reversing, adding, deleting, updating, and slicing list elements.

Theory: Lists can be traversed using loops such as for. Sorting can be done using sort() or sorted() and reversing using reverse().

Lab Program 5: Iterate over a list using a for loop.

```
numbers = [10,20,30,40]
for n in numbers:
    print(n)
```

Lab Program 6: Sort a list using sort() and sorted().

```
numbers = [5,2,9,1]
numbers.sort()
print(numbers)
print(sorted(numbers))
```

Practical Example 5: Print each element of a list.

```
nums = [1,2,3,4]
for i in nums:
    print(i)
```

Practical Example 6: Insert elements into an empty list using append().

```
my_list = []
for i in range(5):
    my_list.append(i)
print(my_list)
```

4. Tuple

Definition: A tuple is an ordered collection similar to a list but immutable, meaning its values cannot be changed after creation. Tuples are written using parentheses ().

Theory: Tuples support operations such as concatenation, repetition, and membership.

Lab Program 7: Create a tuple with multiple data types.

```
t = (10,"Python",3.5,True)
print(t)
```

Lab Program 8: Concatenate two tuples.

```
t1 = (1,2,3)
t2 = (4,5,6)
print(t1 + t2)
```

Practical Example 7: Convert a list into a tuple.

```
my_list = [1,2,3]
t = tuple(my_list)
print(t)
```

Practical Example 8: Create a tuple with multiple data types.

```
tup = (10,"Hello",5.5)
print(tup)
```

Practical Example 9: Concatenate two tuples.

```
a = (1,2)
b = (3,4)
print(a+b)
```

Practical Example 10: Access first index value.

```
t = (10,20,30)
print(t[0])
```

5. Accessing Tuples

Definition: Tuple elements can be accessed using indexing and slicing similar to lists.

Theory: Positive indexing starts from 0 and negative indexing from -1. Slicing retrieves multiple elements.

Lab Program 9: Access values between index 1 and 5.

```
t = (10,20,30,40,50,60,70)
print(t[1:5])
```

Lab Program 10: Access alternate values between index 1 and 5.

```
t = (10,20,30,40,50,60)
print(t[1:5:2])
```

Practical Example 11: Access values between index 1 and 5.

```
t = (1,2,3,4,5,6)
print(t[1:5])
```

Practical Example 12: Access last index value.

```
t = (10,20,30,40)
print(t[-1])
```

6. Dictionaries

Definition: A dictionary is a collection of key-value pairs where each key is unique and used to access its corresponding value.

Theory: Dictionary elements can be added, accessed, updated, or deleted. Important methods include keys(), values(), and items().

Lab Program 11: Create a dictionary with 6 key-value pairs.

```
student = {
    "name": "Arbaz",
    "age": 20,
    "course": "Python",
    "city": "Delhi",
    "id": 101,
    "grade": "A"
}
print(student)
```

Lab Program 12: Access values using dictionary keys.

```
print(student["name"])
print(student["course"])
```

Practical Example 13: Create a dictionary of 6 key-value pairs.

```
data = {"a":1,"b":2,"c":3,"d":4,"e":5,"f":6}
print(data)
```

Practical Example 14: Access values using keys.

```
print(data["a"])
```

7. Working with Dictionaries

Definition: Dictionaries can be iterated using loops and used for tasks such as merging lists or counting characters.

Theory: Using loops with dictionaries and combining lists into dictionaries.

Lab Program 13: Update a value in a dictionary.

```
student["age"] = 21
print(student)
```

Lab Program 14: Merge two lists into a dictionary.

```
keys = ["name", "age", "city"]
values = ["Ali", 22, "Delhi"]
d = {}
for i in range(len(keys)):
    d[keys[i]] = values[i]
print(d)
```

Practical Example 15: Update value at a particular key.

```
d = {"a":1, "b":2}
d["a"] = 10
print(d)
```

Practical Example 16: Separate keys and values.

```
print(d.keys())
print(d.values())
```

Practical Example 17: Convert two lists into a dictionary.

```
k = ["x", "y", "z"]
v = [1, 2, 3]
d = {}
for i in range(len(k)):
    d[k[i]] = v[i]
print(d)
```

Practical Example 18: Count characters in a string.

```
text = "hello"
count = {}
for ch in text:
    count[ch] = count.get(ch, 0) + 1
print(count)
```

8. Functions

Definition: A function is a block of reusable code used to perform a specific task.

Theory: Functions can be created with or without parameters and with or without return values. Lambda functions are anonymous functions.

Lab Program 15: Function that takes a string and prints it.

```
def show(text):  
    print(text)  
  
show("Hello Python")
```

Lab Program 16: Simple calculator using functions.

```
def add(a,b):  
    return a+b  
  
print(add(5,3))
```

Practical Example 19: Print string using function.

```
def message():  
    print("Welcome to Python")  
message()
```

Practical Example 20: Parameterized function for sum.

```
def sum(a,b):  
    print(a+b)  
  
sum(4,6)
```

Practical Example 21: Lambda function with one expression.

```
square = lambda x: x*x  
print(square(5))
```

Practical Example 22: Lambda function with two expressions.

```
add = lambda a,b: a+b  
print(add(10,20))
```

9. Modules

Definition: A module is a file containing Python code (functions and variables) that can be reused in different programs.

Theory: Python provides built-in modules such as math and random. Modules are imported using the import keyword.

Lab Program 17: Import math module and use functions.

```
import math
print(math.sqrt(16))
print(math.factorial(5))
```

Lab Program 18: Generate random numbers using random module.

```
import random
print(random.randint(1,100))
```

Practical Example 23: Demonstrate math module.

```
import math
print(math.pow(2,3))
```

Practical Example 24: Generate random numbers between 1 and 100.

```
import random
print(random.randint(1,100))
```